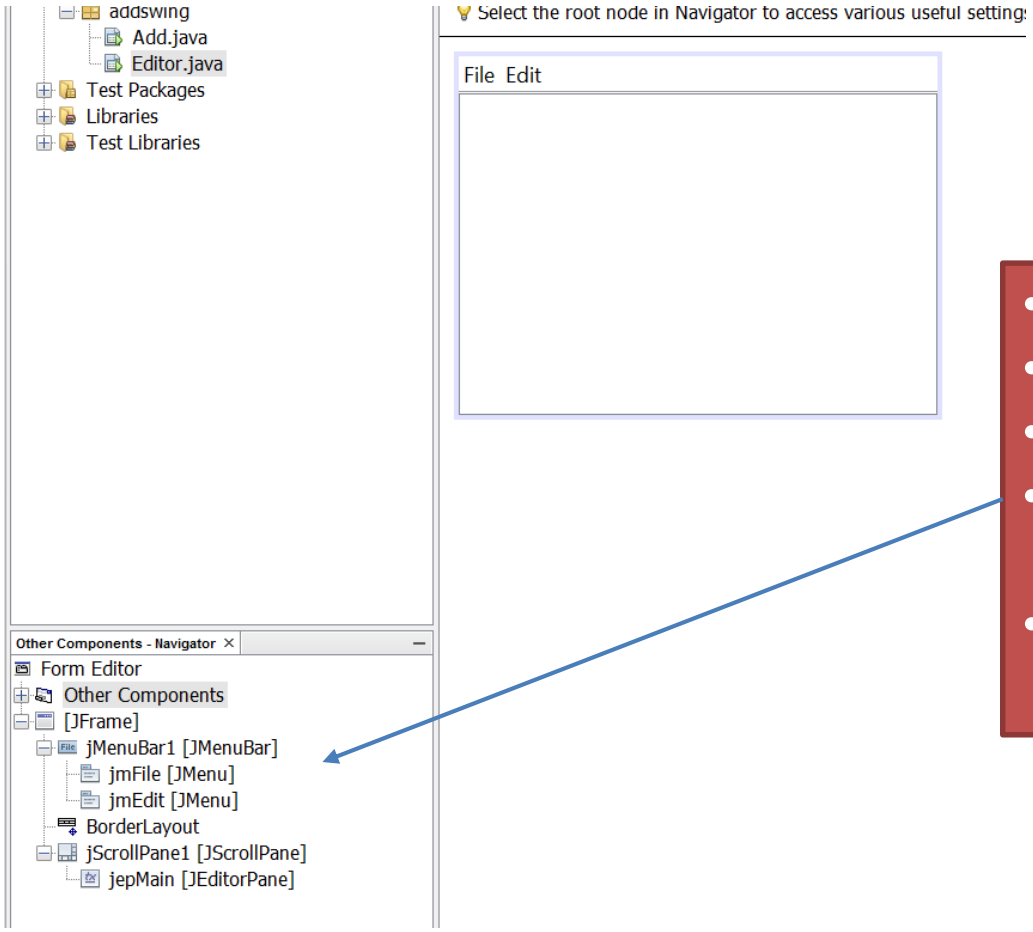
Un semplice editor



- Selezionare il BorderLayout
- Aggiungere una menu bar
- Aggiungere due menu (File, Edit)
- Aggiungere un JScrollPane
  - aggiungere un JEditorPane
- Aggiungere una toolbar a NORTH del frame

Un semplice editor (code)

```
private Map<Object, Action> editorActionTable;

private void initActionTable() {
    editorActionTable = new HashMap<>();
    Action[] actions = jepMain.getActions();
    for (Action action : actions) {
        editorActionTable.put(action.getValue(Action.NAME), action);
    }
}

private Action getEditorActionByName(String action) {
    return editorActionTable.get(action);
}

protected UndoManager undo = new UndoManager();

protected UndoAction undoAction = new UndoAction("Undo");

protected RedoAction redoAction = new RedoAction("Redo");

protected class MyUndoableEditListener implements UndoableEditListener {

    @Override
    public void undoableEditHappened(UndoableEditEvent e) {
        undo.addEdit(e.getEdit());
    }

}
```

Un semplice editor (code)

```
private Map<Object, Action> editorActionTable;

private void initActionTable() {
    editorActionTable = new HashMap<>();
    Action[] actions = jepMain.getActions();
    for (Action action : actions) {
        editorActionTable.put(action.getValue(Action.NAME), action);
    }
}

private Action getEditorActionByName(String action) {
    return editorActionTable.get(action);
}
```

```
protected UndoManager undo = new UndoManager();
```

- Creazione di una Map contenente le azioni di default dell'editor

```
protected RedoAction redoAction = new RedoAction("Redo");

protected class MyUndoableEditListener implements UndoableEditListener {

    @Override
    public void undoableEditHappened(UndoableEditEvent e) {
        undo.addEdit(e.getEdit());
    }

}
```

Un semplice editor (code)

```
private Map<Object, Action> editorActionTable;

private void initActionTable() {
    editorActionTable = new HashMap<>();
    Action[] actions = jepMain.getActions();
    for (Action action : actions) {
        editorActionTable.put(action.getValue(Action.NAME), action);
```

- UndoManager gestisce le operazioni di undo/redo
- UndoAction e RedoAction sono due nuove Action che andremo a definire
- MyUndoableEventListener è il listener che gestisce gli eventi di undo/redo

```
protected UndoManager undo = new UndoManager();

protected UndoAction undoAction = new UndoAction("Undo");

protected RedoAction redoAction = new RedoAction("Redo");

protected class MyUndoableEditListener implements UndoableEditListener {

    @Override
    public void undoableEditHappened(UndoableEditEvent e) {
        undo.addEdit(e.getEdit());
    }

}
```

Un semplice editor (action)

```
protected class UndoAction extends AbstractAction {

    public UndoAction(String name) {
        super(name);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            undo.undo();
        } catch (CannotUndoException ex) {
            JOptionPane.showMessageDialog(null, ex.getMessage(), "Undo exception", JOptionPane.ERROR_MESSAGE);
        }
    }

}

protected class RedoAction extends AbstractAction {

    public RedoAction(String name) {
        super(name);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            undo.redo();
        } catch (CannotRedoException ex) {
            JOptionPane.showMessageDialog(null, ex.getMessage(), "Redo exception", JOptionPane.ERROR_MESSAGE);
        }
    }

}
```

```
private void init() {
    initActionTable();
    undoAction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Undo16.gif"));
    undoAction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Undo24.gif"));
    jmEdit.add(undoAction);
    jToolBar1.add(undoAction);
    redoAction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Redo16.gif"));
    redoAction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Redo24.gif"));
    jmEdit.add(redoAction);
    jToolBar1.add(redoAction);
    jmEdit.add(new JSeparator());
    jToolBar1.add(new JSeparator());
    Action cutaction = getEditorActionByName(DefaultEditorKit.cutAction);
    cutaction.putValue(Action.NAME, "Cut");
    cutaction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Cut16.gif"));
    cutaction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Cut24.gif"));
    jmEdit.add(cutaction);
    jToolBar1.add(cutaction);
    Action copyaction = getEditorActionByName(DefaultEditorKit.copyAction);
    copyaction.putValue(Action.NAME, "Copy");
    copyaction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Copy16.gif"));
    copyaction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Copy24.gif"));
    jmEdit.add(copyaction);
    jToolBar1.add(copyaction);
    Action pasteaction = getEditorActionByName(DefaultEditorKit.pasteAction);
    pasteaction.putValue(Action.NAME, "Cut");
    pasteaction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Paste16.gif"));
    pasteaction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Paste24.gif"));
    jmEdit.add(pasteaction);
    jToolBar1.add(pasteaction);

    Keymap keymap = jepMain.getKeymap();
    KeyStroke keydown = KeyStroke.getKeyStroke(KeyEvent.VK_N, Event.CTRL_MASK);
    keymap.addActionForKeyStroke(keydown, getEditorActionByName(DefaultEditorKit.downAction));
    KeyStroke keyup = KeyStroke.getKeyStroke(KeyEvent.VK_P, Event.CTRL_MASK);
    keymap.addActionForKeyStroke(keyup, getEditorActionByName(DefaultEditorKit.upAction));

    jepMain.getDocument().addUndoableEditListener(new MyUndoableEditListener());
}
```

Un semplice editor (action)

```
private void init() {
    initActionTable();
    undoAction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Undo16.gif"));
    undoAction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Undo24.gif"));
    jmEdit.add(undoAction);
    jToolBar1.add(undoAction);
    redoAction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Redo16.gif"));
    redoAction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Redo24.gif"));
    jmEdit.add(redoAction);
    jToolBar1.add(redoAction);
    jmEdit.add(new JSeparator());
    jToolBar1.add(new JSeparator());
```

- Inizializzazione della tabella delle action di default dell'editor
- Modifica delle proprietà delle action (SMALL/LARGE ICON)
- Aggiungere le action ai menu e alla toolbar

Un semplice editor (action)

```
jToolBar1.add(cutAction);
Action copyaction = getEditorActionByName(DefaultEditorKit.copyAction);
copyaction.putValue(Action.NAME, "Copy");
copyaction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Copy16.gif"));
copyaction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Copy24.gif"));
jmEdit.add(copyaction);
jToolBar1.add(copyaction);
Action pasteaction = getEditorActionByName(DefaultEditorKit.pasteAction);
pasteaction.putValue(Action.NAME, "Cut");
pasteaction.putValue(Action.SMALL_ICON, new ImageIcon("img/general/Paste16.gif"));
pasteaction.putValue(Action.LARGE_ICON_KEY, new ImageIcon("img/general/Paste24.gif"));
```

- Modifica delle proprietà delle action (NAME, SMALL e LARGE ICON) delle action di default dell'editor

Un semplice editor (key stroke)

```
    Keymap keymap = jepMain.getKeymap();
    KeyStroke keydown = KeyStroke.getKeyStroke(KeyEvent.VK_N, Event.CTRL_MASK);
    keymap.addActionForKeyStroke(keydown, getEditorActionByName(DefaultEditorKit.downAction));
    KeyStroke keyup = KeyStroke.getKeyStroke(KeyEvent.VK_P, Event.CTRL_MASK);
    keymap.addActionForKeyStroke(keyup, getEditorActionByName(DefaultEditorKit.upAction));

    jepMain.getDocument().addUndoableEditListener(new MyUndoableEditListener());
}
```

- Aggiunge delle nuove azioni collegate alla pressione di alcune combinazioni di tasti
- In questo caso colleghiamo la pressione di alcune combinazioni di tasti ad eventi di default dell'editor

Un semplice editor (undo/redo…)

```
    Keymap keymap = jepMain.getKeymap();
    KeyStroke keydown = KeyStroke.getKeyStroke(KeyEvent.VK_N, Event.CTRL_MASK);
    keymap.addActionForKeyStroke(keydown, getEditorActionByName(DefaultEditorKit.downAction));
    KeyStroke keyup = KeyStroke.getKeyStroke(KeyEvent.VK_P, Event.CTRL_MASK);
    keymap.addActionForKeyStroke(keyup, getEditorActionByName(DefaultEditorKit.upAction));

    jepMain.getDocument().addUndoableEditListener(new MyUndoableEditListener());
}
```

- Aggiunge il listener per il undo/redo al Document dell'editor pane

DocumentListener

- Document notifica tutti i DocumentListener associati ad esso
  - inserimenti
  - rimozioni
  - modifica dello stile
- Aggiunta di un DocumentListener ad un document
  - `doc.addDocumentListener(new MyDocumentListener());`
- CaretListener permette di ricevere tutti gli eventi di modifica del cursore, deve essere aggiunto ad un JTextComponent

DocumentListener

```
protected class MyDocumentListener implements DocumentListener {

    public void insertUpdate(DocumentEvent e) {
        displayEditInfo(e);
    }

    public void removeUpdate(DocumentEvent e) {
        displayEditInfo(e);
    }

    public void changedUpdate(DocumentEvent e) {
        displayEditInfo(e);
    }

    private void displayEditInfo(DocumentEvent e) {
        Document document = (Document) e.getDocument();
        int changeLength = e.getLength();
        System.out.println(e.getType().toString() + ": "
                + changeLength + " character"
                + ((changeLength == 1) ? ". " : "s. ")
                + " Text length = " + document.getLength()
                + ".");
    }
}
```

DocumentListener

```
protected class MyDocumentListener implements DocumentListener {

    public void init() {
        jmEdit.add(undoAction);
        jmEdit.add(redoAction);
        jmEdit.add(new JSeparator());
        editorActionMap = createActionTable(jepMain);
        jmEdit.add(getActionByName(DefaultEditorKit.cutAction));
        jmEdit.add(getActionByName(DefaultEditorKit.copyAction));
        jmEdit.add(getActionByName(DefaultEditorKit.pasteAction));

        InputMap inputMap = jepMain.getInputMap();
        KeyStroke key1 = KeyStroke.getKeyStroke(KeyEvent.VK_N, Event.CTRL_MASK);
        inputMap.put(key1, DefaultEditorKit.downAction);
        KeyStroke key2 = KeyStroke.getKeyStroke(KeyEvent.VK_P, Event.CTRL_MASK);
        inputMap.put(key2, DefaultEditorKit.upAction);

        jepMain.getDocument().addUndoableEditListener(new MyUndoableEditListener());
        jepMain.getDocument().addDocumentListener(new MyDocumentListener());
    }
            + " Text length = " + document.getLength()
            + ".");
    }
}
```

Un semplice editor (Open File)

- Inserire un MenuItem nel Menu File e rinominarlo in jmiOpen
- Aggiungere l'evento ActionPerformed a jmiOpen

```
private void jmiOpenActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser chooser = new JFileChooser();
    chooser.setMultiSelectionEnabled(false);
    chooser.setFileSelectionMode(JFileChooser.FILES_ONLY);
    if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        File file = chooser.getSelectedFile();
        try {
            BufferedReader reader = new BufferedReader(new FileReade
            StringBuilder sb = new StringBuilder();
            while (reader.ready()) {
                sb.append(reader.readLine()).append("\n");
            }
            reader.close();
            jepMain.setText(sb.toString());
        } catch (IOException ioex) {
            JOptionPane.showMessageDialog(this, "Error to load file:\n" + ioex.getMessage(),
                    "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

JFileChooser apre una finestra di dialogo per la selezione di file o directory

Un semplice editor (Open File)

- Lettura del contenuto testuale dal file
  - BufferedReader+StringBuilder
- Utilizzare il metodo setText di JEditorPane per impostare il testo

```
    private void jmiOpenActionPerformed(java.awt.event.ActionEvent evt) {
        JFileChooser chooser = new JFileChooser();
        chooser.setMultiSelectionEnabled(false);
        chooser.setFileSelectionMode(JFileChooser.FILES_ONLY);
        if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            File file = chooser.getSelectedFile();
            try {
                BufferedReader reader = new BufferedReader(new FileReader(file));
                StringBuilder sb = new StringBuilder();
                while (reader.ready()) {
                    sb.append(reader.readLine()).append("\n");
                }
                reader.close();
                jepMain.setText(sb.toString());
            } catch (IOException ioex) {
                JOptionPane.showMessageDialog(this, "Error to load file:\n" + ioex.getMessage(),
                        "Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }
```